

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

on

Database Management Systems (23DC3PCDBM)

Submitted by

CHANDANA H Y (1BM24IC017)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (IoT & Cybersecurity including Blockchain)



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

August-2025 to December-2025

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

(IoT & Cybersecurity including Blockchain)



CERTIFICATE

This is to certify that the Lab work entitled “Database Management Systems (23IC3PCDBM)” carried out by **Chandana H Y (1BM24IC017)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering (IoT & Cybersecurity including Blockchain)** of the Visvesvaraya Technological University, Belgaum during the year 2025. The Lab report has been approved as it satisfies the academic requirements in respect of a Database Management Systems (23IC3PCDBM) work prescribed for the said degree.

Name of the Lab-Incharge : **Dr. Manjunath D R**

Designation : Associate Professor Professor

Department of CSE(ICB)

BMSCE, Bengaluru

Dr. Prasad G R

Professor and Head

Department of CSE(ICB)

BMSCE, Bengaluru

Index

Sl.No.	Date	Experiment Title	Page No.
1	29/09/25	Insurance Database	4-5
2	13/10/25	Sailor Database	5-9
3	03/11/25	Supplier Database	10-14
4	10/11/25	Employee Database	15-19
5	17/11/25	More Queries on Employee Database	20-21
6	24/11/25	Movie Database	22-27
7	02/12/25	More Queries on Movie Database	28-31

NAME: CHANDANA H Y

USN: 1BM24IC017

1) Add the required constraints on the created tables.

```
mysql> desc sailor;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| sid   | int           | NO   | PRI | NULL    |       |
| sname | varchar(30)   | YES  |     | NULL    |       |
| rating | int          | YES  |     | NULL    |       |
| age   | double        | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

```
mysql> desc boat;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| bid   | int           | NO   | PRI | NULL    |       |
| bname | varchar(30)   | YES  |     | NULL    |       |
| colour | varchar(20)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> desc reserves;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| sid   | int  | YES  | MUL | NULL    |       |
| bid   | int  | YES  | MUL | NULL    |       |
| day   | date | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

2) Populate the relations with at least 5 tuples each.

```
mysql> select * from sailor;
+-----+-----+-----+-----+
| sid | sname   | rating | age |
+-----+-----+-----+-----+
| 120 | joe     | 10     | 28  |
| 121 | adam    | 9      | 30  |
| 122 | smith   | 9      | 26  |
| 123 | forty   | 8      | 29  |
| 124 | lovelace | 10     | 31  |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> select * from boat;
+-----+-----+-----+
| bid | bname   | colour |
+-----+-----+-----+
| 4   | alpha   | red     |
| 5   | beta    | blue    |
| 6   | gamma   | pink     |
| 7   | delta   | orange  |
| 8   | omega   | white   |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> select * from reserves;
+-----+-----+-----+
| sid | bid | day       |
+-----+-----+-----+
| 120 | 4   | 2022-03-22 |
| 121 | 5   | 2022-04-28 |
| 122 | 6   | 2022-02-18 |
| 123 | 7   | 2022-02-11 |
| 124 | 8   | 2022-12-11 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

2)

3) Select names and ages of all sailors. Rename same as 'Sailor Name'

```
mysql> select sname as Sailor_Name,age from sailor;
+-----+-----+
| Sailor_Name | age |
+-----+-----+
| joe         | 28  |
| adam        | 30  |
| smith       | 26  |
| forty       | 29  |
| lovelace    | 31  |
+-----+-----+
5 rows in set (0.00 sec)
```

4) Find all sailors with a rating above 7

5) Find the sid of sailors who have reserved a red boat

```
mysql> alter table sailor rename column sname to Sailor_Name ;
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> select * from sailor where rating>7;
+-----+-----+-----+-----+
| sid | Sailor_Name | rating | age |
+-----+-----+-----+-----+
| 120 | joe         | 10     | 28  |
| 121 | adam        | 9       | 30  |
| 122 | smith       | 9       | 26  |
| 123 | forty       | 8       | 29  |
| 124 | lovelace    | 10      | 31  |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> select sid from reserves where bid in (select bid from boat where colour='red');
+-----+
| sid |
+-----+
| 120 |
+-----+
1 row in set (0.00 sec)
```

6) Find the colors of boats reserved by 'joe'

```
mysql> select colour from boat where bid in (select reserves.bid from sailor inner join reserves on
sailor.sid=reserves.sid where Sailor_Name='joe');
+-----+
| colour |
+-----+
| red    |
+-----+
1 row in set (0.00 sec)
```

7) Delete all boats which have never been reserved.

1 row in set (0.00 sec)

```
mysql> insert into sailor values(125,'max',7,28);  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> insert into boat values(09,'trail','blue');  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> insert into reserves values(125,09,NULL);  
Query OK, 1 row affected (0.00 sec)
```



```
mysql> select * from boat;
```

bid	bname	colour
4	alpha	red
5	beta	blue
6	gamma	pink
7	delta	orange
8	omega	white
9	trail	blue

6 rows in set (0.00 sec)

```
mysql> delete from reserves where day is NULL;  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from reserves;
```

sid	bid	day
120	4	2022-03-22
121	5	2022-04-28
122	6	2022-02-18
123	7	2022-02-11
124	8	2022-12-11

5 rows in set (0.00 sec)

```
mysql> delete from boat where bid not in (select bid from reserves);  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from boat;
```

bid	bname	colour
4	alpha	red
5	beta	blue
6	gamma	pink
7	delta	orange
8	omega	white

5 rows in set (0.00 sec)

NAME: CHANDANA HY
USN: 1BM24IC017

1) Add the required constraints on the created tables.

```
mysql> desc supplier;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| sid   | int           | NO   | PRI | NULL    |       |
| sname | varchar(30)   | YES  |     | NULL    |       |
| address | varchar(50) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> desc part;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| pid   | int           | NO   | PRI | NULL    |       |
| pname | varchar(30)   | YES  |     | NULL    |       |
| colour | varchar(15) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> desc catalog;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| sid   | int           | YES  | MUL | NULL    |       |
| pid   | int           | YES  | MUL | NULL    |       |
| cost  | double        | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

2) Populate the relations with at least 5 tuples each.

```
mysql> select * from supplier;
```

sid	sname	address
120	shyam	bangalore
121	kumar	mangalore
122	swetha	delhi
123	rakesh	mumbai
124	deep	chennai

```
5 rows in set (0.00 sec)
```

```
mysql> select * from part;
```

pid	pname	colour
1230	air pipe	red
1231	car paint	black
1232	car paint	blue
1233	car paint	red
1234	ecu	green
1235	leather	blue
1236	indicator	yellow
1237	headlight	blue
1238	headlight	green
1239	headlight	yellow
1240	headlight	white

```
11 rows in set (0.00 sec)
```

```
mysql> desc catalog;
+-----+-----+-----+-----+-----+-----+
| Field | Type   | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| sid   | int    | YES  | MUL | NULL     |       |
| pid   | int    | YES  | MUL | NULL     |       |
| cost  | double | YES  |     | NULL     |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```



```
mysql> select * from catalog;
+-----+-----+-----+
| sid | pid | cost |
+-----+-----+-----+
| 120 | 1230 | 1000 |
| 120 | 1234 | 1300 |
| 121 | 1234 | 1500 |
| 122 | 1239 | 2200 |
| 122 | 1240 | 2100 |
| 123 | 1238 | 2000 |
| 123 | 1237 | 2200 |
| 124 | 1235 | 3200 |
+-----+-----+-----+
8 rows in set (0.00 sec)
```

3) Select the ID and names of all the suppliers.

```
mysql> select sid,sname from supplier;
+-----+-----+
| sid | sname |
+-----+-----+
| 120 | shyam |
| 121 | kumar |
| 122 | swetha |
| 123 | rakesh |
| 124 | deep |
+-----+-----+
5 rows in set (0.00 sec)
```

4) Select the most costly part available in the catalog.

5) Find the names of parts for which there is some supplier.

```
mysql> select * from part where pid in(select pid from catalog where cost=(select max(cost) from catalog));
+-----+-----+-----+
| pid | pname | colour |
+-----+-----+-----+
| 1235 | leather | blue |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> select * from part where pid in (select distinct pid from catalog);
+-----+-----+-----+
| pid | pname | colour |
+-----+-----+-----+
| 1230 | air pipe | red |
| 1234 | ecu | green |
| 1235 | leather | blue |
| 1237 | headlight | blue |
| 1238 | headlight | green |
| 1239 | headlight | yellow |
| 1240 | headlight | white |
+-----+-----+-----+
7 rows in set (0.00 sec)
```

6) Find the sids of suppliers who supply a red part and a green part.

```
mysql> select distinct c1.sid
-> from catalog c1
-> join part p1 on c1.pid=p1.pid
-> join catalog c2 on c2.sid=c1.sid
-> join part p2 on c2.pid=p2.pid
-> where p1.colour='red' and p2.colour='green';
+-----+
| sid |
+-----+
| 120 |
+-----+
1 row in set (0.00 sec)
```

7) Delete all parts of a given ID.

```
mysql> delete from part where pid in (select pid from catalog where sid=121);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select * from catalog;
```

sid	pid	cost
120	1230	1000
122	1239	2200
122	1240	2100
123	1238	2000
123	1237	2200
124	1235	3200

```
6 rows in set (0.00 sec)
```

```
mysql> select * from part;
```

pid	pname	colour
1230	air pipe	red
1231	car paint	black
1232	car paint	blue
1233	car paint	red
1235	leather	blue
1236	indicator	yellow
1237	headlight	blue
1238	headlight	green
1239	headlight	yellow
1240	headlight	white

```
10 rows in set (0.00 sec)
```

NAME : CHANDANA H Y
USN : 1BM24IC017

1. Using Scheme diagram, Create tables by properly specifying the primary keys and the foreign keys.

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
EMPNO	int	NO	PRI	NULL	
ENAME	varchar(50)	YES		NULL	
HIREDATE	date	YES		NULL	
SAL	decimal(7,2)	YES		NULL	
DEPTNO	int	YES	MUL	NULL	
MGR_NO	int	YES	MUL	NULL	

6 rows in set (0.00 sec)

```
mysql> desc dept;
```

Field	Type	Null	Key	Default	Extra
DEPTNO	int	NO	PRI	NULL	
DNAME	varchar(50)	YES		NULL	
DLOC	varchar(50)	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> desc incentives;
```

Field	Type	Null	Key	Default	Extra
EMPNO	int	NO	PRI	NULL	
INCENTIVE_DATE	date	NO	PRI	NULL	
INCENTIVE_AMOUNT	decimal(7,2)	YES		NULL	

3 rows in set (0.00 sec)

2. Enter greater than five tuples for each table.

```
mysql> select* from employee;
```

EMPNO	ENAME	HIREDATE	SAL	DEPTNO	MGR_NO
7001	KING	1981-11-17	5000.00	10	NULL
7002	BLAKE	1982-05-01	3000.00	20	7001
7003	CLARK	1982-06-09	3000.00	10	7001
7004	JONES	1983-04-02	2500.00	20	7002
7005	SMITH	1983-12-17	2000.00	20	7002
7006	FORD	1984-01-23	2800.00	20	7002
7007	ADAMS	1984-05-23	2200.00	10	7003
7008	SCOTT	1985-07-09	2900.00	10	7003
7009	MILLER	1986-08-12	4000.00	50	7003
7010	DAVIS	1983-03-03	2900.00	20	7001
7011	EVANS	1984-09-14	1800.00	20	7010

```
11 rows in set (0.01 sec)
```

```
mysql> select* from dept;
```

DEPTNO	DNAME	DLOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
50	MARKETING	DALLAS
60	IT	NEW YORK

```
6 rows in set (0.01 sec)
```

```
mysql> select* from project;
```

PNO	PNAME	PLOC
1001	ERP System	DALLAS
1002	Web Portal	CHICAGO
1003	Audit Tool	NEW YORK
1004	Data Warehouse	BOSTON
1005	Ad Campaign	DALLAS
1006	Security Audit	NEW YORK


```
mysql> select* from incentives;
```

EMPNO	INCENTIVE_DATE	INCENTIVE_AMOUNT
7002	2019-01-10	1000.00
7003	2019-03-05	400.00
7004	2019-01-20	300.00
7004	2019-01-25	200.00
7005	2019-01-15	700.00
7007	2019-02-10	300.00
7008	2018-12-25	500.00

```
7 rows in set (0.01 sec)
```



```
mysql> select* from assigned_to;
```

EMPNO	PNO	JOB_ROLE
7002	1001	Project Manager
7003	1003	Project Manager
7004	1001	Developer
7005	1001	Analyst
7007	1003	Tester
7008	1002	Lead Developer
7009	1005	Ad-Hoc

```
7 rows in set (0.01 sec)
```

3. List the name of the managers with the maximum employees

```
mysql> with mcount as(select mgr_no,count(*) as employeecount
-> from employee where mgr_no is not null
-> group by mgr_no)
-> select employee.ename from employee
-> join mcount on employee.empno=mcount.mgr_no
-> where mcount.employeecount=(select max(employeecount) from mcount);
```

ename
KING
BLAKE
CLARK

```
3 rows in set (0.00 sec)
```

4. Display those managers name whose salary is more than average salary of his employee.

```
mysql> select m.ename as manager,m.sal as manager_salary,avg(e.sal) as avg_em
p_salary
-> from employee m join employee e
-> group by m.empno,m.ename,m.sal having m.sal>avg(e.sal);
```

manager	manager_salary	avg_emp_salary
MILLER	4000.00	2918.181818
CLARK	3000.00	2918.181818
BLAKE	3000.00	2918.181818
KING	5000.00	2918.181818

```
4 rows in set (0.00 sec)
```

5. Find the name of the second top level managers of each department.

```
mysql> select m.ename,d.dname
-> from employee m join dept d on m.deptno=d.deptno
-> where m.mgr_no=(select empno from employee where mgr_no is null)
-> and m.empno in (select distinct mgr_no from employee
-> where mgr_no is not null);
```

ename	dname
BLAKE	RESEARCH
CLARK	ACCOUNTING
DAVIS	RESEARCH

```
3 rows in set (0.00 sec)
```

6. Find the employee details who got second maximum incentive in January 2019.

```
mysql> select * from employee where empno in (
    -> select empno from incentives where incentive_amount=(
    -> select max(incentive_amount) from incentives where
    -> incentive_amount<(select max(incentive_amount) from incentives)));
```

EMPNO	ENAME	HIREDATE	SAL	DEPTNO	MGR_NO
7005	SMITH	1983-12-17	2000.00	20	7002

1 row in set (0.00 sec)

7. Display those employees who are working in the same department where his manager is working.

```
mysql> select e.ename as employee_name,m.ename as manager_name,d.dname
    -> as department from employee e join employee m
    -> on e.mgr_no=m.empno
    -> join dept d on e.deptno=d.deptno where e.deptno=m.deptno;
```

employee_name	manager_name	department
CLARK	KING	ACCOUNTING
JONES	BLAKE	RESEARCH
SMITH	BLAKE	RESEARCH
FORD	BLAKE	RESEARCH
ADAMS	CLARK	ACCOUNTING
SCOTT	CLARK	ACCOUNTING
EVANS	DAVIS	RESEARCH

7 rows in set (0.00 sec)

NAME : CHANDANA H Y
USN : 1BM24IC017

1. List the employees who have worked on all projects located in 'New York'.

```
mysql> SELECT EMPLOYEE.ENAME FROM EMPLOYEE JOIN ASSIGNED_TO WHERE  
-> PNO = ALL (SELECT PNO FROM PROJECT WHERE PLOC='NEW YORK');  
Empty set (0.00 sec)
```

2. Find the department with the highest total salary of its employees.

```
mysql> SELECT D.DEPTNO,D.DNAME,D.DLOC,SUM(E.SAL) AS TOTAL_SAL  
-> FROM EMPLOYEE E JOIN DEPT D ON E.DEPTNO=D.DEPTNO  
-> GROUP BY D.DEPTNO,DNAME,D.DLOC HAVING  
-> SUM(E.SAL)>=ALL (SELECT SUM(SAL) FROM EMPLOYEE  
-> GROUP BY DEPTNO);  
+-----+-----+-----+-----+  
| DEPTNO | DNAME   | DLOC   | TOTAL_SAL |  
+-----+-----+-----+-----+  
|      20 | RESEARCH | DALLAS | 15000.00 |  
+-----+-----+-----+-----+  
1 row in set (0.02 sec)
```

3. Display the employees who have received incentives greater than 50% of their salary.

```
mysql> SELECT * FROM EMPLOYEE WHERE EMPNO IN (SELECT EMPLOYEE.EMPNO FROM EMPLOYEE JOIN INCENTIVES ON EMPLOYEE.EMPNO=INCENTIVES.EMPNO WHERE INCENTIVE_AMOUNT> (SAL*0.5));  
Empty set (0.00 sec)
```

4. List the project names and the number of employees working on each project.

5. Find the name of the employee with the earliest hire date in each department.

```
mysql> SELECT * FROM EMPLOYEE E WHERE HIREDATE=  
-> (SELECT MIN(HIREDATE) FROM EMPLOYEE WHERE DEPTNO=E.DEPTNO);  
+-----+-----+-----+-----+-----+-----+  
| EMPNO | ENAME   | HIREDATE | SAL      | DEPTNO | MGR_NO |  
+-----+-----+-----+-----+-----+-----+  
| 7001 | KING    | 1981-11-17 | 5000.00 | 10     | NULL   |  
| 7002 | BLAKE   | 1982-05-01 | 3000.00 | 20     | 7001   |  
| 7009 | MILLER  | 1986-08-12 | 4000.00 | 50     | 7003   |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)
```

6. List all employees who are not assigned to any project.

```
mysql> SELECT * FROM EMPLOYEE WHERE EMPNO NOT IN
-> (SELECT DISTINCT EMPNO FROM ASSIGNED_TO);
```

EMPNO	ENAME	HIREDATE	SAL	DEPTNO	MGR_NO
7001	KING	1981-11-17	5000.00	10	NULL
7006	FORD	1984-01-23	2800.00	20	7002
7010	DAVIS	1983-03-03	2900.00	20	7001
7011	EVANS	1984-09-14	1800.00	20	7010

```
4 rows in set (0.00 sec)
```

7. Identify departments where all employees earn a salary greater than the average salary of all employees.

```
mysql> SELECT D.DEPTNO,D.DNAME,D.DLOC,MIN(E.SAL) AS MIN_SAL,
-> COUNT(E.EMPNO) AS NUM_EMPLOYEES FROM EMPLOYEE E
-> JOIN DEPT D ON E.DEPTNO=D.DEPTNO
-> GROUP BY D.DEPTNO,D.DNAME,D.DLOC
-> HAVING MIN(E.SAL)>(SELECT AVG(SAL) FROM EMPLOYEE);
```

DEPTNO	DNAME	DLOC	MIN_SAL	NUM_EMPLOYEES
50	MARKETING	DALLAS	4000.00	1

```
1 row in set (0.01 sec)
```

NAME: CHANDANA H Y
USN: 1BM24IC017

```
mysql> DESC ACTOR;
```

Field	Type	Null	Key	Default	Extra
act_id	int	NO	PRI	NULL	
act_name	varchar(30)	YES		NULL	
Act_gender	char(1)	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> DESC DIRECOR;
```

ERROR 1146 (42S02): Table 'movie.direcor' doesn't exist

```
mysql> DESC DIRECTOR;
```

Field	Type	Null	Key	Default	Extra
DIR_ID	int	NO	PRI	NULL	
DIR_NAME	varchar(30)	YES		NULL	
DIR_PHONE	varchar(10)	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> DESC MOVIES;
```

Field	Type	Null	Key	Default	Extra
MOV_ID	int	NO	PRI	NULL	
MOV_TITLE	varchar(30)	YES		NULL	
MOV_YEAR	int	YES		NULL	
MOV_LANG	varchar(15)	YES		NULL	
DIR_ID	int	YES	MUL	NULL	

5 rows in set (0.00 sec)

```
mysql> DESC MOVIE_CAST;
```

Field	Type	Null	Key	Default	Extra
ACT_ID	int	YES	MUL	NULL	
MOV_ID	int	YES	MUL	NULL	
ROLE	varchar(30)	YES		NULL	

```
3 rows in set (0.00 sec)
```

```
mysql> DESC RATING;
```

Field	Type	Null	Key	Default	Extra
MOV_ID	int	NO	PRI	NULL	
REV_STARS	decimal(2,1)	YES		NULL	

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM ACTOR;
```

act_id	act_name	Act_gender
101	WILL	M
112	TAYLOR	F
120	PETER	M
128	SMITH	M
211	JULES	F
212	JOHN	M
223	DAISY	F

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM DIRECTOR;
```

DIR_ID	DIR_NAME	DIR_PHONE
333	HANNAH	8720385836
567	NICK	9725725732
876	JACK	8572635464

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM MOVIES;
```

MOV_ID	MOV_TITLE	MOV_YEAR	MOV_LANG	DIR_ID
101	The Silent Horizon	2019	English	876
102	Shadows of Tomorrow	2021	English	567
103	Beyond the Northern Star	2018	French	333
104	Whispering Winds	2020	Spanish	876
105	Rise of the Forgotten	2022	English	567
106	Echoes in the Valley	2017	Hindi	333
107	The Last Ember	2023	English	876
108	Voyage of the Dawn Mirage	2024	Japanese	567

```
8 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM MOVIE_CAST;
```

ACT_ID	MOV_ID	ROLE
120	101	Lead Protagonist
128	101	Antagonist
223	101	Supporting Actor
112	102	Lead Hero
212	102	Scientist
101	102	Villain
211	103	Explorer
120	103	Captain
128	103	Crew Member
223	104	Detective
112	104	Witness
212	104	Journalist
101	105	Commander
211	105	Rebel Leader
120	105	Advisor
128	106	Historian
223	106	Archaeologist
112	107	Lead Warrior
212	108	Navigator
101	108	Merchant

```
20 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM RATING;
```

MOV_ID	REV_STARS
101	8.3
102	7.9
103	8.1
104	7.4
105	8.7
106	7.2
107	8.9
108	7.8

- i. List the titles of all movies directed by 'Hitchcock'.(USED HANNAH INSTEAD)

```
mysql> SELECT MOV_TITLE FROM MOVIES WHERE DIR_ID=(SELECT DIR_ID FROM DIRECTOR
WHERE DIR_NAME='HANNAH');
+-----+
| MOV_TITLE |
+-----+
| Beyond the Northern Star |
| Echoes in the Valley |
+-----+
2 rows in set (0.00 sec)
```

- ii. Find the movie names where one or more actors acted in two or more movies.

```
mysql> SELECT DISTINCT M.Mov_Title
-> FROM MOVIES M
-> JOIN MOVIE_CAST MC ON M.Mov_id = MC.Mov_id
-> WHERE MC.Act_id IN (
->     SELECT Act_id
->     FROM MOVIE_CAST
->     GROUP BY Act_id
->     HAVING COUNT(Mov_id) >= 2
-> );
+-----+
| Mov_Title |
+-----+
| The Silent Horizon |
| Shadows of Tomorrow |
| Beyond the Northern Star |
| Whispering Winds |
| Rise of the Forgotten |
| Echoes in the Valley |
| The Last Ember |
| Voyage of the Dawn Mirage |
+-----+
8 rows in set (0.00 sec)
```

iii. List all actors who acted in a movie before 2000 and also in a movie after 2015
(use JOIN operation)

```
mysql> SELECT * FROM ACTOR WHERE ACT_ID IN (SELECT ACT_ID FROM MOVIE_CAST C JOIN MOVIES M ON C.MOV_ID=M.MOV_ID WHERE MOV_YEAR<2000 AND MOV_YEAR>2015);
Empty set (0.00 sec)
```

iv. Find the title of movies and number of stars for each movie that has at least one rating and find the highest number of stars that movie received. Sort the result by movie title.

```
mysql> SELECT MOVIES.MOV_TITLE, RATING.REV_STARS, (SELECT MAX(REV_STARS) FROM RATING WHERE RATING.MOV_ID=MOVIES.MOV_ID) AS HIGHEST FROM MOVIES JOIN RATING
-> ON MOVIES.MOV_ID=RATING.MOV_ID ORDER BY MOV_TITLE;
+-----+-----+-----+
| MOV_TITLE | REV_STARS | HIGHEST |
+-----+-----+-----+
| Beyond the Northern Star | 8.1 | 8.1 |
| Echoes in the Valley | 7.2 | 7.2 |
| Rise of the Forgotten | 8.7 | 8.7 |
| Shadows of Tomorrow | 7.9 | 7.9 |
| The Last Ember | 8.9 | 8.9 |
| The Silent Horizon | 8.3 | 8.3 |
| Voyage of the Dawn Mirage | 7.8 | 7.8 |
| Whispering Winds | 7.4 | 7.4 |
+-----+-----+-----+
8 rows in set (0.00 sec)
```

v. Update rating of all movies directed by 'Steven Spielberg' to 5.(USED NICK INSTEAD)

```
mysql> UPDATE RATING SET REV_STARS=5 WHERE MOV_ID IN (SELECT MOV_ID FROM
-> MOVIES JOIN DIRECTOR ON MOVIES.DIR_ID=DIRECTOR.DIR_ID WHERE
-> DIR_NAME='NICK');
Query OK, 3 rows affected (0.00 sec)
Rows matched: 3 Changed: 3 Warnings: 0

mysql> SELECT * FROM RATING;
+-----+-----+
| MOV_ID | REV_STARS |
+-----+-----+
| 101 | 8.3 |
| 102 | 5.0 |
| 103 | 8.1 |
| 104 | 7.4 |
| 105 | 5.0 |
| 106 | 7.2 |
| 107 | 8.9 |
| 108 | 5.0 |
+-----+-----+
8 rows in set (0.00 sec)
```

NAME: CHANDANA HY
USN: 1BM24IC017

List the names of actors who have never received a rating below 4 in any of their movies.

```
mysql> select act_name from actor where act_id in (select act_id from
-> movie_cast where mov_id not in (select mov_id from rating where
-> rev_stars<4)) and act_id not in (select act_id from movie_cast
-> where mov_id in(select mov_id from rating where rev_stars<4));
+-----+
| act_name |
+-----+
| Leonardo DiCaprio |
| Christian Bale |
| Matt Damon |
| Robert Downey Jr. |
| Emma Watson |
| Samuel L. Jackson |
+-----+
6 rows in set (0.00 sec)
```

Find the titles of movies which have at least twice the average rating of all movies released in the same language.

```
mysql> select movies.mov_title,movies.mov_lang,avg(rating.rev_stars) as movie
_avg_rating,(select avg(rating.rev_stars) from movies join rating on movies.m
ov_id=rating.mov_id where movies.mov_lang=movies.mov_lang) as lang_avg_rating
from movies join rating on movies.mov_id=rating.mov_id group by movies.mov_i
d,movies.mov_title,movies.mov_lang having avg(rating.rev_stars)>=2*(select av
g(rating.rev_stars) from movies join rating on movies.mov_id=rating.mov_id wh
ere movies.mov_lang=movies.mov_lang);
Empty set (0.00 sec)
```

For each director, find the year in which they directed the highest number of movies, and how many movies they directed in that year.

```
mysql> select director.dir_name,counts.mov_year,counts.movie_count from direc
tor join (select movies.dir_id,movies.mov_year,count(*) as movie_count from m
ovies group by movies.dir_id,movies.mov_year) as counts on director.dir_id=co
unts.dir_id where counts.movie_count=(select max(c2.movie_count) from (select
movies.dir_id,movies.mov_year,count(*) as movie_count from movies group by m
ovies.dir_id,movies.mov_year) as c2 where c2.dir_id=counts.dir_id) order by d
irector.dir_name,counts.mov_year;
```

dir_name	mov_year	movie_count
Christopher Nolan	1985	1
Christopher Nolan	2005	1
Christopher Nolan	2010	1
Christopher Nolan	2014	1
Martin Scorsese	2006	1
Michael Bay	2007	1
Quentin Tarantino	1994	1
Steven Spielberg	1993	1
Steven Spielberg	2000	1
Steven Spielberg	2002	1
Steven Spielberg	2005	1

```
11 rows in set (0.00 sec)
```

List all actors who have acted in movies with at least 10 different co-actors (i.e., distinct other actors appearing with them in at least one common movie).

```
mysql> select actor.act_name,count(distinct actor2.act_id) as number_of_coact
ors from actor join movie_cast on actor.act_id=movie_cast.act_id join movie_c
ast as movie_cast2 on movie_cast.mov_id=movie_cast2.mov_id and movie_cast.act
_id<>movie_cast2.act_id join actor as actor2 on movie_cast2.act_id=actor2.act
_id group by actor.act_id,actor.act_name having count(distinct actor2.act_id)
>=10 order by number_of_coactors desc;
```

Empty set (0.00 sec)

Find the actors who have worked with both 'Christopher Nolan' and 'Steven Spielberg' as directors, but have never worked with any director whose name starts with 'M'.

```
mysql> select actor.act_name from actor where actor.act_id in (select movie_cast.act_id from movie_cast join movies on movie_cast.mov_id=movies.mov_id join director on movies.dir_id=director.dir_id where director.dir_name='christopher nolan') and actor.act_id in (select movie_cast.act_id from movie_cast join movies on movie_cast.mov_id=movies.mov_id join director on movies.dir_id=director.dir_id where director.dir_name='steven spielberg') and actor.act_id not in (select movie_cast.act_id from movie_cast join movies on movie_cast.mov_id=movies.mov_id join director on movies.dir_id=director.dir_id where director.dir_name like 'm%');
+-----+
| act_name |
+-----+
| Tom Hanks |
| Scarlett Johansson |
+-----+
2 rows in set (0.00 sec)
```

For each actor, compute their “career span” in years, defined as (latest_movie_year – earliest_movie_year + 1).

List only those actors whose career span is at least 15 years and who have acted in at least 3 movies.

```
mysql> select actor.act_name,min(movies.mov_year) as earliest_year,max(movies.mov_year) as latest_year,(max(movies.mov_year)-min(movies.mov_year)+1) as career_span,count(distinct movies.mov_id) as movie_count from actor join movie_cast on actor.act_id=movie_cast.act_id join movies on movie_cast.mov_id=movies.mov_id group by actor.act_id,actor.act_name having career_span>=15 and movie_count>=3;
+-----+-----+-----+-----+-----+
| act_name | earliest_year | latest_year | career_span | movie_count |
+-----+-----+-----+-----+-----+
| Leonardo DiCaprio | 1985 | 2014 | 30 | 6 |
| Tom Hanks | 1993 | 2010 | 18 | 5 |
| Scarlett Johansson | 1993 | 2010 | 18 | 4 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Find the top 3 most “versatile” actors, where versatility is defined as the number of distinct roles (Role) they have played across all movies.

```
mysql> select act_name,distinct_role_count from (select actor.act_name,count(distinct movie_cast.role) as distinct_role_count from actor join movie_cast on actor.act_id=movie_cast.act_id group by actor.act_id,actor.act_name) as rolecounts where distinct_role_count>=(select distinct_role_count from (select count(distinct movie_cast.role) as role_count from actor join movie_cast on actor.act_id=movie_cast.act_id group by actor.act_id order by role_count desc limit 1 offset 2) as thirdplace) order by distinct_role_count desc,act_name asc;
+-----+-----+
| act_name | distinct_role_count |
+-----+-----+
| Leonardo DiCaprio | 6 |
| Tom Hanks | 4 |
| Morgan Freeman | 2 |
| Natalie Portman | 2 |
| Robert Downey Jr. | 2 |
| Samuel L. Jackson | 2 |
+-----+-----+
6 rows in set (0.00 sec)
```

For each director and language combination (Dir_Name, Mov_Lang), find the average rating of their movies in that language, and the total number of movies they directed in that language.

```
mysql> select director.dir_name,movies.mov_lang,count(distinct movies.mov_id) as
movie_count,avg(rating.rev_stars) as avg_rating from director join movies on
director.dir_id=movies.dir_id join rating on movies.mov_id=rating.mov_id group
by director.dir_name,movies.mov_lang having count(distinct movies.mov_id)>=2 an
d avg(rating.rev_stars)>4;
```

dir_name	mov_lang	movie_count	avg_rating
Christopher Nolan	English	4	5.00000

1 row in set (0.00 sec)

List only those director–language pairs where the director has directed at least 2 movies in that language and the average rating is strictly greater than 3.

```
mysql> select movies.mov_title,movies.mov_year,avg(rating.rev_stars) as avg_rat
ing,min(rating.rev_stars) as min_rating,max(rating.rev_stars) as max_rating fro
m movies join rating on movies.mov_id=rating.mov_id group by movies.mov_id,movi
es.mov_title,movies.mov_year having avg(rating.rev_stars)>=3 and min(rating.rev
_stars)<=3;
```

mov_title	mov_year	avg_rating	min_rating	max_rating
Jurassic Park	1993	3.00000	3.0	3.0

1 row in set (0.00 sec)

Find movies that are “critically inconsistent”: movies whose average rating is at least 4.5 but which also have at least one rating of 2 or less.

```
mysql> select director.dir_name,movies.mov_title,movies.mov_year,avg(rating.rev
_stars) as avg_rating from director join movies on director.dir_id=movies.dir_i
d join rating on movies.mov_id=rating.mov_id group by director.dir_id,director.
dir_name,movies.mov_id,movies.mov_title,movies.mov_year having count(distinct m
ovies.mov_id)=1 and avg(rating.rev_stars)>=4.5;
```

dir_name	mov_title	mov_year	avg_rating
Christopher Nolan	Inception	2010	5.00000
Christopher Nolan	Interstellar	2014	5.00000
Christopher Nolan	Epic Saga	1985	5.00000
Christopher Nolan	Epic Return	2005	5.00000
Steven Spielberg	French Classic	2000	5.00000
Martin Scorsese	The Departed	2006	5.00000
Quentin Tarantino	Pulp Fiction	1994	5.00000

7 rows in set (0.00 sec)